

# Universitatea Tehnica din Cluj-Napoca

Facultatea de Automatica si Calculatoare

---

## Documentatie Proiect: Scena 3D Interactivă

Sisteme de Prelucrare Grafica (SPG)

---

*Autor:*

Stolniceanu Iustin-Pavel

*Profesor coordonator:*

Constantin Nandra

Ianuarie 2026

# Cuprins

|          |                                              |          |
|----------|----------------------------------------------|----------|
| <b>1</b> | <b>Prezentarea temei</b>                     | <b>1</b> |
| <b>2</b> | <b>Scenariul</b>                             | <b>1</b> |
| 2.1      | Descrierea scenei si a obiectelor            | 1        |
| 2.2      | Functionalitati                              | 2        |
| <b>3</b> | <b>Detalii de implementare</b>               | <b>3</b> |
| 3.1      | Functii si algoritmi                         | 3        |
| 3.1.1    | 1. Randarea vegetatiei: Tehnica Billboarding | 3        |
| 3.1.2    | 2. Detectia coliziunilor: Algoritmul AABB    | 3        |
| 3.1.3    | 3. Animatia camerei: Catmull-Rom Spline      | 3        |
| 3.2      | Modelul grafic                               | 4        |
| 3.2.1    | 1. Umbre dinamice: Shadow Mapping            | 4        |
| 3.2.2    | 2. Modelul de iluminare Phong                | 5        |
| 3.2.3    | 3. Ceata pe Skybox                           | 5        |
| 3.3      | Structuri de date                            | 6        |
| 3.4      | Ierarhia de clase                            | 7        |
| <b>4</b> | <b>Manual de utilizare</b>                   | <b>8</b> |
| <b>5</b> | <b>Concluzii și dezvoltări ulterioare</b>    | <b>8</b> |
| 5.1      | Dezvoltări ulterioare                        | 8        |

# 1 Prezentarea temei

Acest proiect are ca scop realizarea unei scene 3D fotorealiste si interactive, dezvoltata in C++ utilizand API-ul grafic OpenGL 4.1. Arhitectura aplicatiei se bazeaza pe o suita de biblioteci specializate: **GLFW** pentru managementul ferestrei si a input-ului, **GLEW** pentru gestionarea extensiilor OpenGL si **GLM** (OpenGL Mathematics) pentru operatii vectoriale si matriceale.

Proiectul demonstreaza stapanirea conceptelor fundamentale din grafica computerizata, de la managementul resurselor (modele, texturi, shadere) la implementarea unor tehnici avansate precum:

- Iluminare complexa bazata pe modelul Phong, incluzand surse de lumina multiple (directionala, punctiforma si spotlight).
- Randarea umbrelor dinamice pentru lumina directionala prin tehnica *Shadow Mapping*.
- Optimizarea randarii pentru scene complexe, folosind tehnica *billboarding* pentru vegetatie.
- Managementul camerei la persoana intai, cu detectie de coliziuni si o traiectorie cinematica pre-calculata (Catmull-Rom spline).
- Efecte de post-procesare simple, precum ceata (fog).

Obiectivul principal a fost crearea unui mediu virtual populat (o padure generata procedural), care sa permita explorarea libera si interactiunea cu diverse efecte dinamice, demonstrand o intelegere practica a pipeline-ului grafic modern.

## 2 Scenariul

### 2.1 Descrierea scenei si a obiectelor

Scena este compusa dintr-un teren vast, populat cu vegetatie si diverse obiecte 3D importate:

- **Terenul:** Un plan texturat extins pentru a simula solul [1].
- **Vegetatia:** O padure compusa din peste 200 de copaci generati dinamic, folosind texturi de tip billboard [2].
- **Obiecte:** Nanosuit [3], Mario [4], Creeper [5], Zombie [6], Bonfire [7] și un Cort [8].
- **Skybox:** Un cubemap care ofera context vizual cerului, utilizand o textura dintr-un pachet de resurse Minecraft [9].



Figura 1: Vizualizarea scenei in conditii de iluminare globala (zi).



Figura 2: Vizualizarea scenei cu lanterna (spotlight) activata pe timp de noapte.

## 2.2 Functionalitati

- **Controlul Camerei:** Deplasare WASD si orientare prin mouse.
- **Iluminare Interactiva:** Comutare lanterna (tasta F).
- **Animatie Cinematica:** Traiectorie fluida bazata pe o curba spline [10].
- **Moduri de Vizualizare:** Solid, wireframe si polygonal points.

## 3 Detalii de implementare

### 3.1 Functii si algoritmi

În această secțiune sunt detaliate provocările tehnice întâmpinate și soluțiile algoritmice implementate pentru a asigura atât realismul vizual, cât și performanța necesară unei execuții în timp real.

#### 3.1.1 1. Randarea vegetatiei: Tehnica Billboarding

**Problema:** Crearea unei păduri dense (peste 200 de copaci) folosind modele 3D High-Poly ar fi dus la o scădere drastică a numărului de cadre pe secundă (FPS).

**Rezolvarea mea:** Am utilizat tehnica *Billboarding*. Fiecare copac este reprezentat prin 3 planuri texturate care se intersectează la un unghi de  $60^\circ$ . Această structură oferă iluzia de volum din orice unghi de vizualizare orizontal, utilizând doar 12 vertex-uri.

#### 3.1.2 2. Detectia coliziunilor: Algoritmul AABB

**Problema:** Navigarea într-o scenă populată necesită un sistem care să împiedice camera să treacă prin obiecte solide.

**Rezolvarea mea:** Am implementat volume de coliziune de tip *Axis-Aligned Bounding Box* (AABB). Verificarea coliziunii se reduce la o serie de comparații între coordonatele camerei și limitele cutiei, rezultând într-o complexitate de  $O(1)$ .

#### 3.1.3 3. Animația camerei: Catmull-Rom Spline

**Problema:** Definirea unei traiectorii automate fluide pentru cameră, care să treacă prin puncte de interes predefinite în scenă.

**Rezolvarea mea:** Am implementat o traiectorie fluidă bazată pe curba *Catmull-Rom Spline* [10]. Această tehnică este ideală pentru animații cinematice deoarece garantează că traiectoria va trece exact prin punctele de control specificate, generând o mișcare naturală și fără discontinuități.

Curba este definită parametric, calculând un punct  $\mathbf{P}(t)$  pentru un segment determinat de patru puncte de control consecutive:  $\mathbf{P}_0$ ,  $\mathbf{P}_1$ ,  $\mathbf{P}_2$  și  $\mathbf{P}_3$ . Segmentul de curbă efectiv este desenat între  $\mathbf{P}_1$  și  $\mathbf{P}_2$ , în timp ce  $\mathbf{P}_0$  și  $\mathbf{P}_3$  sunt utilizate pentru a calcula tangențele la capetele segmentului, asigurând astfel continuitatea  $C1$  între segmente adiacente.

Formula matematică pentru un punct pe curbă este:  $P(t) = 0.5 \times ((2 \cdot P_1) + (-P_0 + P_2) \cdot t + (2 \cdot P_0 - 5 \cdot P_1 + 4 \cdot P_2 - P_3) \cdot t^2 + (-P_0 + 3 \cdot P_1 - 3 \cdot P_2 + P_3) \cdot t^3)$

Unde 't' este un parametru care variază de la 0 la 1 și controlează poziția de-a lungul segmentului de curbă între  $\mathbf{P}_1$  și  $\mathbf{P}_2$ .

În implementarea mea, am definit un set de puncte de control (vectori 'glm::vec3') în scenă. Funcția 'getSplinePoint' primește ca intrare timpul global al animației 't' și lista de puncte de control.

1. **Identificarea segmentului curent:** Timpul global 't' este normalizat pentru a determina indexul primului punct de control al segmentului curent ( $\mathbf{p}_1$ ). Celelalte puncte ( $\mathbf{p}_0$ ,  $\mathbf{p}_2$ ,  $\mathbf{p}_3$ ) sunt calculate relativ la  $\mathbf{p}_1$ , asigurând continuitatea la capetele listei de puncte (traiectorie în buclă).

2. **Calcularea parametrului local:** Partea fracționară a lui 't' devine parametrul local de interpolare pe segmentul curent.
3. **Interpolarea:** Se aplică formula Catmull-Rom pentru a calcula coordonatele 3D exacte ale camerei pentru acel moment în timp. Poziția rezultată este apoi folosită pentru a actualiza matricea de vizualizare a camerei, creând iluzia unei mișcări continue și line.

---

```
glm::vec3 getSplinePoint(float t, const std::vector<glm::vec3>& controlPoints) {
    int p0, p1, p2, p3;

    // Asigura o traiectorie in bucla (closed loop)
    p1 = (int)t % controlPoints.size();
    p2 = (p1 + 1) % controlPoints.size();
    p3 = (p2 + 1) % controlPoints.size();
    // Punctul anterior lui p1
    p0 = (p1 - 1 + controlPoints.size()) % controlPoints.size();

    // Parametrul local `t` intre p1 si p2
    float local_t = t - (int)t;

    // Formula Catmull-Rom
    float t2 = local_t * local_t;
    float t3 = t2 * local_t;

    glm::vec3 point = 0.5f * (
        (2.0f * controlPoints[p1]) +
        (-controlPoints[p0] + controlPoints[p2]) * local_t +
        (2.0f * controlPoints[p0] - 5.0f * controlPoints[p1] + 4.0f * controlPoints[p2] - controlPoints[p3]) * t2 +
        (-controlPoints[p0] + 3.0f * controlPoints[p1] - 3.0f * controlPoints[p2] + controlPoints[p3]) * t3
    );

    return point;
}
```

---

## 3.2 Modelul grafic

Randarea scenei se bazeaza pe pipeline-ul programabil OpenGL (GLSL 4.10), orchestrat printr-o serie de programe de tip shader pentru a obtine efecte vizuale complexe.

### 3.2.1 1. Umbre dinamice: Shadow Mapping

Implementarea umbrelor pentru lumina direțională (soarele) respecta algoritmul standard de *Shadow Mapping* [11]. Procesul este impartit in doua etape distincte (pass-uri) pentru fiecare cadru randat:

1. **Depth Pass:** Scena este randata din perspectiva sursei de lumina. In acest pas, nu se calculeaza culorile finale, ci doar informatia de adancime (distanța de la lumina la cel mai apropiat fragment). Aceste valori sunt scrise intr-o textura speciala, numita *depth map*. Pentru a controla spatiul de vizualizare al luminii, se utilizeaza o proiectie ortografica. O provocare specifica, shadow acne; a fost mitigata prin tehnica *front-face culling* in acest pass, care asigura ca se randeaza fetele din spate ale obiectelor, eliminand problemele de auto-umbrire.

2. **Lighting Pass:** Scena este randata din perspectiva camerei. In fragment shader, pentru fiecare fragment, se calculeaza pozitia sa din perspectiva luminii (folosind matricea *light space*). Aceasta pozitie este apoi folosita pentru a esantiona din *depth map*. Comparand adancimea stocata in textura cu adancimea curenta a fragmentului, se poate determina daca fragmentul este in umbra (daca adancimea sa este mai mare decat cea din harta) sau iluminat direct.

### 3.2.2 2. Modelul de iluminare Phong

Iluminarea este calculata per-fragment folosind modelul Phong [12], care simuleaza interactiunea luminii cu suprafetele prin combinarea a trei componente:

- **Ambientă:** Simulează lumina globală, indirectă, asigurând că și zonele neiluminate direct nu sunt complet negre.
- **Difuză:** Reprezintă reflexia luminii uniform în toate direcțiile, a cărei intensitate depinde de unghiul dintre normala suprafeței și direcția luminii.
- **Speculară:** Simulează reflexiile concentrate (luciul), a căror intensitate depinde de poziția camerei, direcția luminii și normala suprafeței.

Acest model a fost implementat pentru a gestiona trei tipuri de surse de lumină, care pot fi combinate în scenă:

- **Lumină direcțională:** O sursă de lumină aflată la infinit (ex: soarele), ale cărei raze sunt paralele.
- **Lumină punctiformă (Point Light):** O sursă care emite lumină uniform în toate direcțiile dintr-un singur punct.
- **Lanternă (Spotlight):** O sursă de lumină care emite un con de lumină într-o direcție specifică. Este atașată de cameră și are o margine atenuată (penumbra) pentru un efect realist, calculată cu funcția `smoothstep` în GLSL între unghiul interior și cel exterior al conului.

### 3.2.3 3. Ceata pe Skybox

Pentru a integra vizual skybox-ul cu restul scenei, în special în condiții de ceață, a fost necesară aplicarea unui efect similar și pe acesta. Deoarece skybox-ul este redat ca fiind la o distanță infinită, nu se poate folosi adâncimea pentru a calcula intensitatea ceții.

Soluția implementată în fragment shader-ul skybox-ului ('skyboxShader.frag') utilizează coordonata verticală a texturii ('textureCoordinates.y') pentru a modula efectul. Ideea este că ceața este mai densă la orizont (unde 'y' este aproape de 0) și inexistentă în zenit (la polul superior al cubemap-ului, unde 'y' este 1).

1. Se calculează un factor de ceață ('factor') pornind de la '1.0 - abs(textureCoordinates.y)'. Acest calcul returnează 1.0 la orizont și scade spre 0 la poli.
2. Factorul este ridicat la o putere pentru a controla finețea tranziției și este multiplicat cu densitatea globală a ceții, asigurând consistența cu ceața aplicată pe restul obiectelor.

3. Culoarea finală este un amestec (interpolare liniară cu ‘mix’) între culoarea originală a skybox-ului și culoarea ceții, pe baza factorului calculat.

---

```
#version 410 core

in vec3 textureCoordinates;
out vec4 color;

uniform samplerCube skybox;
uniform float fogDensity;
vec4 fogColor = vec4(0.5f, 0.5f, 0.5f, 1.0f);

void main()
{
    // Deoarece skybox-ul se afla la o distanta infinita(teoretic), nu putem folosi distanta fata d
    // Folosim coordonata de pe axa Y pentru a determina unde sa aplicam ceata

    // Citim culoarea din cube map
    vec4 skyboxColor = texture(skybox, textureCoordinates, 2.5 /*LOD - Level of detail*/);

    //la orizont vom avea intensitate maxima, la polii cerului vom avea intensitate minima
    float factor = clamp(1.0 - abs(textureCoordinates.y), 0.0, 1.0);

    factor = pow(factor, 1.2/* controleaza cat de lin se face tranzitia*/) * min(fogDensity * 40.0 /

    color = mix(skyboxColor, fogColor, factor);
}
```

---

### 3.3 Structuri de date

Pentru o organizare eficientă a memoriei și a logicii aplicației, am definit structuri dedicate pentru fizică și managementul obiectelor.

#### 1. BoundingBox

Această structură definește un volum de coliziune aliniat axelor (AABB). Este esențială pentru a verifica rapid dacă poziția camerei se intersectează cu un obiect, comparând coordonatele punctului cu vectorii min și max ai cutiei.

---

```
// Simple Axis-Aligned Bounding Box for collision detection
struct BoundingBox {
    glm::vec3 min;
    glm::vec3 max;

    bool contains(const glm::vec3& point) const {
        return (point.x >= min.x && point.x <= max.x) &&
            (point.y >= min.y && point.y <= max.y) &&
            (point.z >= min.z && point.z <= max.z);
    }
};
```

---

#### 2. TreeInstance

Pentru a popula pădurea fără a încărca excesiv procesorul, am folosit o structură lightweight care reține doar datele esențiale transformării fiecărui copac, în loc să duplicăm geometria.

---

```
// Data for a single tree instance in the world
struct TreeInstance {
    glm::vec3 position;
    int typeIndex;    // 0, 1, or 2 (determines which .obj to use)
    float scale;
    float rotationY;
};
```

---

### 3.4 Ierarhia de clase

Arhitectura software a proiectului este construită pe o structură orientată obiect, menită să gestioneze resursele grafice în mod modular. Clasele sunt grupate în namespace-ul **gps** și au următoarele responsabilități:

- **gps::Model3D**: Această clasă reprezintă componenta principală pentru gestionarea obiectelor complexe din scenă. Se ocupă de încărcarea fișierelor externe (format **.obj**) și coordonează aplicarea texturilor prin intermediul bibliotecii *stb\_image*.
- **gps::Mesh**: Definește geometria de bază a unui obiect. Clasa gestionează datele brute trimise către placa video (vârfuri, normale și coordonate de textură) și execută comanda efectivă de desenare a acestora.
- **gps::Objects**: Acționează ca un administrator centralizat al tuturor instanțelor de obiecte din lumea virtuală. Utilizează containere STL (de tip **std::vector**) pentru a stoca eficient transformările (poziție, scalare, rotație) elementelor repetitive, precum vegetația. De asemenea, optimizează procesul de randare prin parcurgerea listelor interne și emiterea organizată a comenzilor către GPU, gestionând totodată și logica de populare procedurală a scenei (generarea pozițiilor copacilor).
- **gps::Camera**: Funcționează ca „ochiul virtual” al utilizatorului în spațiul 3D. Rolul său este de a calcula *matricea de vizualizare*, permițând deplasarea în scenă folosind tastele WASD și orientarea privirii cu ajutorul mouse-ului.
- **gps::Shader**: Gestionează partea programabilă a pipeline-ului grafic. Această clasă încarcă, compilează și activează programele scrise în limbajul GLSL, care sunt executate direct pe procesorul grafic (GPU).
- **gps::SkyBox**: O clasă specializată pentru randarea fundalului. Utilizează un cub de mari dimensiuni și un set de 6 texturi (cubemap) pentru a crea iluzia unui orizont infinit și a unui mediu înconjurător imersiv.

## 4 Manual de utilizare

| Tastă      | Acțiune                         |
|------------|---------------------------------|
| W, A, S, D | Mișcare în scenă                |
| Mouse      | Rotire privire                  |
| F          | Comutare Lanternă (Flashlight)  |
| V          | Pornire/Oprire animație Spline  |
| 3, 4, 5    | Moduri de vizualizare           |
| 7, 8       | Modificare factor de ceață      |
| Q, E       | Rotire robot Nanosuit           |
| J, L       | Mișcare sursă de lumină (soare) |

## 5 Concluzii și dezvoltări ulterioare

Proiectul demonstrează o implementare robustă a unei scene 3D interactive în OpenGL, integrând cu succes concepte cheie de grafică computerizată. Au fost atinse obiectivele principale, precum crearea unui mediu explorabil, implementarea unui sistem de iluminare multi-sursă și optimizarea randării prin tehnici specifice (billboarding, instancing implicit). Implementarea umbrelor dinamice prin shadow mapping pentru lumina principală și a sistemului de coliziuni AABB contribuie la realismul și interactivitatea scenei.

### 5.1 Dezvoltări ulterioare

Deși proiectul actual este funcțional și complet în raport cu cerințele, există numeroase direcții în care poate fi extins pentru a include tehnici mai avansate:

- **Umbre pentru surse de lumină multiple:** O extensie naturală ar fi implementarea umbrelor și pentru sursele de lumină punctiforme și pentru lanternă. Acest lucru ar necesita tehnici mai complexe, precum *Omnidirectional Shadow Mapping* (folosind cubemaps) pentru point lights și randări suplimentare din perspectiva lanternei.
- **Efecte de post-procesare avansate:** Se pot adăuga efecte precum *Bloom* pentru a crea o strălucire intensă în jurul surselor de lumină, *Motion Blur* pentru a spori senzația de viteză sau corecții de culoare pentru a ajusta atmosfera vizuală.
- **Normal Mapping:** Înlocuirea texturilor simple cu hărți de normale (*normal maps*) ar crește exponențial detaliul vizual al suprafețelor fără a adăuga poligoane suplimentare, simulând denivelări și micro-detalii pe obiecte precum terenul sau modelele 3D.
- **Sistem de particule:** Pentru efecte vizuale dinamice, precum fum, scântei sau praf, se poate implementa un sistem de particule gestionat pe GPU.
- **Încărcare de modele asincronă:** Pentru scene foarte mari, încărcarea tuturor resurselor la pornire poate fi lentă. Un sistem care încarcă modelele și texturile pe un fir de execuție separat ar îmbunătăți experiența utilizatorului.

## Bibliografie

- [1] Mojang Studios, “Grass texture,” 2009. [Online]. Available: <https://texture-packs.com/resourcepack/default-pack/>

- [2] Tech Monkey Bussines, “Vegetation billboard texture,” 2026. [Online]. Available: [https://www.techmonkeybusiness.com/galleries/Texture\\_Galleries/Billboard\\_Trees/](https://www.techmonkeybusiness.com/galleries/Texture_Galleries/Billboard_Trees/)
- [3] Crytek, “Nanosuit 3d model,” 2011. [Online]. Available: <https://free3d.com/3d-model/crysis-2-nanosuit-2-97837.html>
- [4] Nintendo, “Mario 3d model,” 2025. [Online]. Available: <https://sketchfab.com/3d-models/mario-ds-fbx-552baccb1ef74a799fb7440fdc5fd69d#download>
- [5] Mojang Studios, “Creeper 3d model,” 2009. [Online]. Available: <https://sketchfab.com/3d-models/minecraft-creeper-c986450b4d884c2c94c0d3168671c543#download>
- [6] —, “Zombie 3d model,” 2009. [Online]. Available: <https://sketchfab.com/3d-models/minecraft-zombie-45037ec52b6a4444a0981f916d0dbd43#download>
- [7] Sketchfab, “Bonfire 3d model,” 2026, accesat la 15 Ianuarie 2026. [Online]. Available: <https://sketchfab.com/3d-models/bonfire-a96d894e8bd14eebaf7cd023178e663>
- [8] —, “Tent 3d model,” 2026, accesat la 15 Ianuarie 2026. [Online]. Available: <https://sketchfab.com/3d-models/tent-9569b78b8d7d4fe1b8d3e8d71ede13d0#download>
- [9] Mojang Studios, “Minecraft skybox texture,” 2011. [Online]. Available: <https://texture-packs.com/resourcepack/default-pack/>
- [10] Wikipedia Contributors, “Catmull-rom spline,” 2026, accesat la 10 Ianuarie 2026. [Online]. Available: [https://en.wikipedia.org/wiki/Catmull%E2%80%93Rom\\_spline](https://en.wikipedia.org/wiki/Catmull%E2%80%93Rom_spline)
- [11] A. Sabou, *Laborator 9: Tehnici de randare a umbrelor (Shadow Mapping)*, Universitatea Tehnica din Cluj-Napoca, 2025, prelucrare Grafica.
- [12] —, *Laborator 7: Iluminare - Modelul Phong*, Universitatea Tehnica din Cluj-Napoca, 2025, prelucrare Grafica.